CASE STUDY · AGENT RUNTIME

HTK Agent Approval

Human-in-the-loop cho AI agent: state machine lưu bền vững, sandbox idempotent, phê duyệt diff và audit đầy đủ.

7/7
run E2E hoàn tất trên bản live

2/2
outbox sau khi tick lại (idempotent)

Tác giả: HTK 0396241674
Ngày: 11/11/2026
Live: htk-agent-approval.vercel.app

1. Bài toán

Một agent có thể nâng phiên bản dependency, đổi feature config hoặc gửi email hàng loạt. Giá trị của tự động hóa nằm ở chỗ con người không phải làm từng thao tác; rủi ro nằm ở chỗ agent hành động khi chưa được phép, hoặc hành động hai lần vì một lần retry.

Bốn kiểu thất bại điển hình

- **Mất trạng thái khi process chết.** Nếu bước đang chạy chỉ nằm trong RAM, restart sẽ không biết đã validate hay chưa, đã gửi email hay chưa.
- **Retry gây tác dụng phụ kép.** Worker thử lại bước apply sau timeout mơ hồ có thể ghi đè hoặc gửi trùng, dù lần đầu đã thành công.
- **Phê duyệt không có bằng chứng.** Một nút "Approve" không kèm diff, risk và lịch sử bước là phê duyệt mù; quyết định không được lưu lại thì không thể điều tra sau.
- **Không có vùng cách ly.** Cho agent chạy lệnh shell hoặc gửi email thật trong demo biến lỗi nhỏ thành sự cố không thể hoàn tác.

Mục tiêu đo được

- Mỗi bước của mỗi run được ghi vào DB với trạng thái, số lần thử, output và thời điểm.
- Worker tuyệt đối không vượt bước `await_approval` nếu chưa có quyết định của con người.
- Apply lại cùng một run không tạo bản ghi thứ hai; bước verify đọc lại dữ liệu để chứng minh.
- Retry tối đa 3 lần, run hết hạn sau 24 giờ, mọi chuyển trạng thái đều có audit.

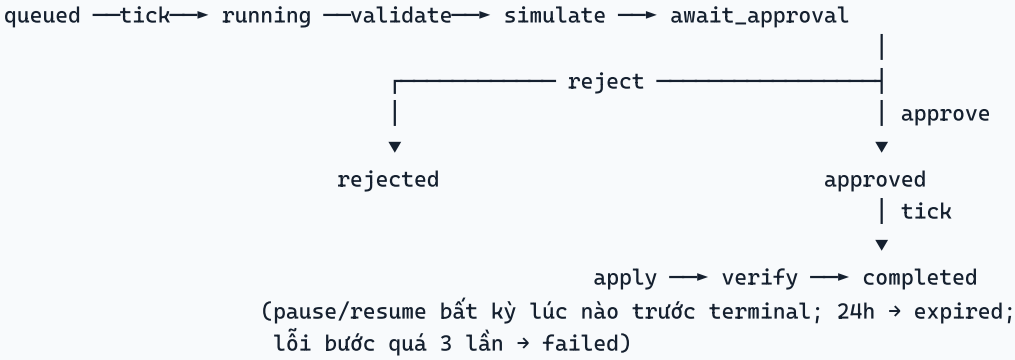
> Đối tượng sử dụng: platform engineer và DevOps cần giao việc cho agent nhưng vẫn giữ quyền quyết định ở các thay đổi có rủi ro. Demo dùng ba loại tác vụ đại diện: bump dependency, đổi config, gửi email hàng loạt.

2. Giải pháp

Ứng dụng là một Next.js full-stack trên Vercel. Mỗi đề xuất là một run có state machine 5 bước, được lưu trong PostgreSQL (PGlite) và snapshot lên Vercel Blob private. Worker là endpoint tick chạy trong Node runtime, không phải process nền thường trực.

```
Browser (workspace)
   │  fetch JSON + session cookie httpOnly
   ▼
Next.js route handlers (Vercel Node, maxDuration 30s)
   ├─ POST /api/runs          planner → diff + risk → run + 5 steps
   ├─ POST /api/runs/action   approve / reject / pause / resume
   ├─ POST /api/worker/tick   advance tối đa 5 run × 6 bước
   ├─ GET  /api/runs          run + steps + output
   └─ GET  /api/audit, /api/metrics
   │
   ▼
PGlite (PostgreSQL WASM)
   runs ──< run_steps            trạng thái từng bước
   runs ──< tool_calls           idempotency_key UNIQUE
   sandbox_state, outbox         vùng áp dụng cô lập
   audit_logs, rate_limits, request_logs
   │  JSON snapshot sau mỗi mutation
   ▼
Vercel Blob private  state/agent-approval.json
```

State machine



Luồng một đề xuất

1. API validate zod, planner kiểm tra nghiệp vụ: semver hợp lệ, 1–5 người nhận, config key thuộc allowlist.
2. Planner tạo unified diff và risk. Major bump hoặc >3 người nhận là high; bulk email là medium.
3. Run và 5 step được insert trong một lần; audit ghi `run_created`.
4. Worker tick: validate → simulate (60 ms trong sandbox DB) → dừng ở `await_approval`, ghi diff vào step output.
5. Người dùng đọc diff và bấm duyệt; quyết định được audit với actor và thời điểm.
6. Tick tiếp: apply kiểm tra step duyệt đã done, ghi sandbox trong transaction kèm idempotency key, verify đọc lại.
7. Nếu verify sai, bước retry tối đa 3 lần rồi run chuyển `failed`; không retry vô hạn.

Quyết định thiết kế

Vì sao lưu từng bước thay vì một cột status?

Một cột status không trả lời được "bước nào đã chạy, mấy lần, output gì". Bảng `run_steps` có unique `(run_id, step)` nên retry cập nhật đúng dòng, còn UI dựng timeline từ dữ liệu thật. Đây cũng là cơ sở để resume sau restart: worker đọc `current_step` và tiếp tục, không chạy lại từ đầu.

Vì sao chặn duyệt trong bước apply, không chỉ ở API?

Nếu chỉ chặn ở UI hoặc endpoint action, một đường gọi worker khác có thể bỏ qua. Bước apply tự query trạng thái `await_approval` và ném lỗi nếu chưa done. Đây là enforcement ở tầng dữ liệu, nơi không thể vô tình bỏ qua.

Vì sao idempotency key nằm trong transaction?

`apply:<runId>` là unique key trong `tool_calls`. Lần apply đầu ghi thay đổi sandbox và key trong cùng transaction; lần sau đọc key và trả về kết quả cũ với cờ `skipped`. Nếu process chết giữa transaction, rollback xóa cả hai, nên không có trạng thái nửa vời.

Vì sao sandbox chỉ ghi DB?

Demo không được phép gửi email thật hay chạy shell trên máy người khác. `sandbox_state` mô phỏng `package.json` và `config.json`, `outbox` mô phỏng email. Verify đọc lại đúng các bảng đó, nên toàn bộ vòng lặp vẫn có bằng chứng mà không có tác dụng phụ ra ngoài.

3. Trade-off & đo lường

Kết quả đo thật (11/11/2026)

Metric	Kết quả	Nguồn
Run E2E hoàn tất	**7/7 completed**, 1 rejected	Luồng script trên bản live
Thứ tự bước	**5/5 done**	GET /api/runs
Idempotency apply	**outbox 2/2** sau khi tick lại	Output bước verify
Approval latency p50 / p95	**2.366 ms / 3.718 ms** (7 mẫu)	GET /api/metrics
Request latency p50 / p95	**145 ms / 5.393 ms** (30 mẫu)	GET /api/metrics
Retry trên luồng thành công	**0**	GET /api/metrics
Rate limit	**429 ở request thứ 10/phút**	Gửi 12 request liên tiếp
Unit test	**7/7 pass**	npm test

Đánh đổi có chủ đích

- **Worker poll thay vì queue managed.** Không cần SQS/Redis và billing, nhưng tick là best-effort: không có backoff lịch tự động, người dùng hoặc cron phải gọi. Với tải thật cần queue + visibility timeout + dead-letter.
- **Planner tất định, không LLM.** Diff và risk tái lập được, test được, không tốn token; đổi lại không nhận yêu cầu ngôn ngữ tự do. Tôi chọn chứng minh runtime an toàn trước, rồi mới thêm lớp hiểu ngôn ngữ.
- **Sandbox DB, không container.** An toàn tuyệt đối nhưng chưa chứng minh thay đổi trên filesystem hay network thật. Adapter apply có thể thay sau mà không đổi state machine.
- **Snapshot JSON lên Blob.** Không cần DB managed; đổi lại ghi đồng thời giữa nhiều lambda là last-write-wins. Etag sync giảm đọc cũ, không loại bỏ hoàn toàn mất ghi.
- **Vercel-only thay vì backend Go/Flask.** Một deployment, một ngôn ngữ; đổi lại worker bị giới hạn 30 giây mỗi invocation và cold start PGlite làm p95 request tăng.

> Chưa hoàn hảo: approval latency đo bằng độ trễ script bấm duyệt, không phải thời gian người thật suy nghĩ; p95 request bị cold start chi phối; chưa benchmark 100+ run đồng thời; chưa có cron tự động gọi worker; dữ liệu demo là 8 run do chính tôi tạo để kiểm thử. Các số trên là đo thật trong phạm vi demo.

Điều gì sẽ làm tiếp

- Thay tick endpoint bằng Vercel Cron + queue có khóa per-run để chạy song song an toàn.
- Thêm chữ ký và TTL cho idempotency key; đo tỷ lệ duplicate khi giả lập timeout giữa transaction.
- Mở rộng sandbox sang container ephemeral, giữ nguyên contract apply/verify.
- Đo approval latency với người thật và phân bố risk theo loại thay đổi.

4. Bài học

1. Trạng thái phải sống lâu hơn process

Bài kiểm tra thật không phải "chạy được một lần", mà là dừng giữa chừng, reload, rồi tiếp tục đúng bước. Việc lưu từng step và đọc lại `current_step` biến resume thành mặc định thay vì tính năng phụ. Nếu chỉ lưu status tổng, worker sẽ phải đoán và dễ chạy lại tác dụng phụ.

2. Idempotency là ràng buộc dữ liệu, không phải lời hứa trong code

Tôi đặt unique key trong bảng và ghi nó cùng transaction với thay đổi. Nhờ vậy, kể cả khi hai lần tick chồng nhau, database mới là nơi phân xử. Kiểm thử live đã xác nhận: tick lại run đã completed vẫn giữ outbox đúng 2 bản ghi, không thành 4.

3. Phê duyệt chỉ có nghĩa khi có bằng chứng đi kèm

Nút approve không đủ. Người duyệt cần diff, risk, output từng bước và lịch sử ai làm gì. Đưa audit log thành bảng riêng giúp trả lời câu hỏi điều tra sau sự cố, đồng thời tạo dữ liệu thật để đo approval latency thay vì ước lượng.

4. Giới hạn phải được viết ra trước khi được hỏi

Worker poll và snapshot Blob là lựa chọn demo-scale. Ghi rõ last-write-wins, giới hạn 30 giây và việc chưa có queue thật quan trọng ngang với việc demo chạy. Người đánh giá cần biết con số nào đáng tin và hệ thống sẽ gãy ở đâu khi tải tăng.